

Project Report

Fundamentals of Database Systems

May 6, 2026

Abstract

This document details the design and implementation of our Pharmacy Management System. This project implements a fully normalized relational database designed to accurately track medical inventory and sales. This projects highlights our entity-relationship modeling, SQL implementation, query optimization strategies. It also shows the security measures applied to ensure data integrity.

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Objectives	3
1.3	Scope	3
2	Database Design	3
2.1	Entity-Relationship Diagram (ERD)	3
2.2	Normalization	4
2.3	Schema Design	4
3	Implementation	4
3.1	Database Tables	4
3.2	SQL Queries	5
3.3	Stored Procedures, Triggers, Transactions	5
4	Query Optimization	5
5	Data Integrity and Security	5
5.1	Constraints and Data Validation	5
5.2	Access Control and Security Measures	6
6	Results and Discussion	6
6.1	Testing and Validation	6
6.2	Challenges and Limitations	6

7 Conclusion	6
---------------------	----------

8 References	6
---------------------	----------

1 Introduction

1.1 Problem Statement

Managing pharmacy inventory requires strict precision. Tracking expiration dates and stock levels manually or with outdated software is inefficient and increases the risk of dispensing expired medications. The goal with this project is to replace manual tracking methods with an automated well-structured backend database.

1.2 Objectives

The core objectives of this project are to:

- Design a relational database to track suppliers, medications, and daily sales.
- Automate quality control using database triggers to reject expired drug entries.
- Ensure and guarantee financial data accuracy during the checkout process using ACID-compliant transactions.
- Optimize database performance for routine tasks like generating monthly revenue reports.

1.3 Scope

This project focuses exclusively on the backend database architecture. We cover Entity-Relationship modeling, database normalization, and advanced SQL programmability (stored procedures and triggers). The development of a front-end graphical user interface falls outside the scope of this implementation.

2 Database Design

2.1 Entity-Relationship Diagram (ERD)

The database consists of six core entities: Suppliers, Drugs, Patients, Prescriptions, Sales, and Sale_Items.

- A Supplier provides one-to-many Drugs.
- A Patient can have zero-to-many Prescriptions and Sales.
- A Sale is composed of one-to-many Sale_Items.
- Drugs and Sales share a many-to-many relationship, resolved by the associative entity Sale Items.

Figure 2.1.1: Entity-Relationship Diagram (Crow's Foot Notation)

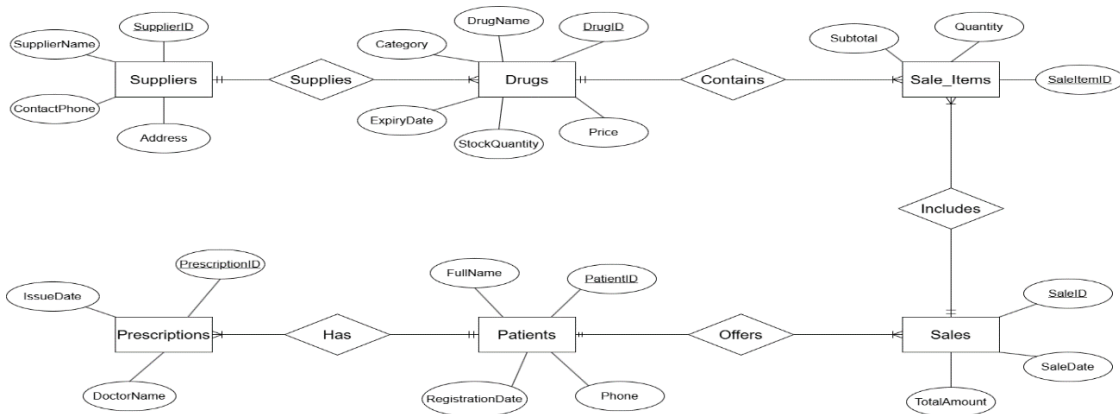
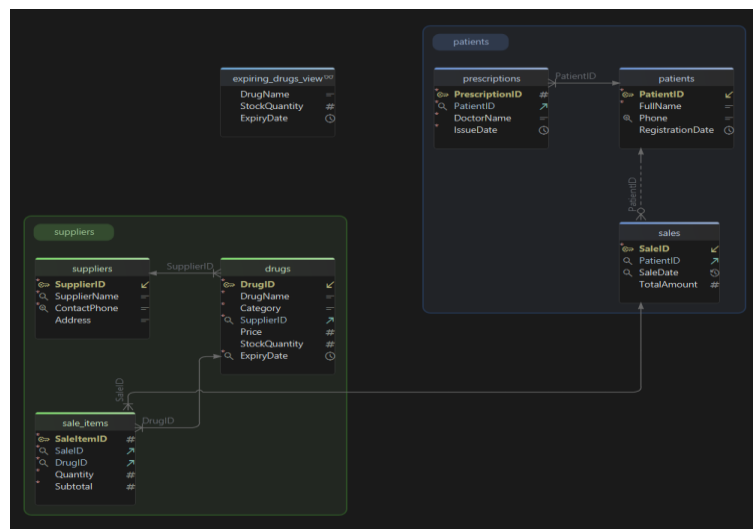


Figure 2.1.2: Entity-Relationship Diagram Implementation



2.2 Normalization

Our database schema strictly adheres to the 3rd Normal Form (3NF) to eliminate data redundancy and stop insertion/deletion anomalies:

- **1NF:** All attributes are atomic, and each table has a designated primary key.
- **2NF:** All non-key attributes entirely depend on the primary key. For example, in the Sale_Items table, attributes rely on the composite logic of both the SaleID & DrugID.
- **3NF:** We removed all transitive dependencies. For instance, supplier contact information is stored in the Suppliers table rather than being repeating alongside every medication in the Drugs table.

2.3 Schema Design

The relational schema leverages INT AUTO_INCREMENT primary keys for all tables to guarantee and ensure unique record identification. Also, implementing strict foreign key constraints; for example, a Supplier cannot be deleted if they have active Drugs in the inventory.

3 Implementation

3.1 Database Tables

The implemented database utilizes six specific tables:

- **Suppliers:** SupplierID (PK), SupplierName, ContactPhone, Address.
- **Drugs:** DrugID (PK), DrugName, Category, Price, StockQuantity, ExpiryDate, SupplierID (FK).
- **Patients:** PatientID (PK), FullName, Phone, RegistrationDate.
- **Prescriptions:** PrescriptionID (PK), PatientID (FK), DoctorName, IssueDate.
- **Sales:** SaleID (PK), PatientID (FK), SaleDate, TotalAmount.
- **Sale Items:** SaleItemID (PK), SaleID (FK), DrugID (FK), Quantity, Subtotal.

3.2 SQL Queries

To handle standard operations, a comprehensive set of queries were developed.

Figure 1: Complex Subquery for Low Stock

```
SELECT DrugName, StockQuantity, Price
FROM Drugs
WHERE StockQuantity < 100
AND SupplierID IN (
    SELECT SupplierID
    FROM Suppliers
    WHERE Address LIKE '%6th of October%'
);
```

Additionally, 'JOIN' queries were crafted to generate comprehensive receipts by linking Sale Items, Sales, and Drugs, alongside 'SUM' and 'COUNT' aggregate functions to calculate monthly revenue. In addition to the transaction-based updates inside Process_Sale, we support direct manual inventory adjustments. For example: **(UPDATE Drugs SET StockQuantity = 150 WHERE DrugID = 5;)** which is useful for restocking or correcting stock after physical count.

3.3 Stored Procedures, Triggers, Transactions

To model real-world pharmacy operations, we integrated advanced SQL Logic:

- **Triggers:** A `BEFORE INSERT` trigger (`Prevent_Expired_Drug_Insert`) validates `NEW.ExpiryDate` against the `CURRENT_DATE`. If the drug is already expired, the trigger stops the insertion and returns an error.
- **Transactions:** The `Process_Sale` stored procedure encapsulates an ACID-compliant transaction. It utilizes `SELECT ... FOR UPDATE` to lock rows during concurrent checkout, deducts the correct inventory, and saves the financial transaction. If inventory is insufficient, it triggers a `ROLLBACK`.

4 Query Optimization

Specific optimizations were applied to ensure the database performs well as data scales:

- **Indexing:** B-Tree indexes were created on `Drugs(ExpiryDate)` and `Sales(SaleDate)` to reduce the I/O cost of range scans during monthly audits and expiration checks.
- **Caching/Views:** A view named `Expiring_Drugs_View` acts as a cached, virtual table. It surfaces the drugs expiring within a 3-month window without requiring the database engine to recalculate complex `DATE_ADD` logic.
- **Execution Plans:** We used the `EXPLAIN` statement on our monthly revenue query to verify the indexing strategy. By switching to a date range query, we eliminated a full table scan and reduced the I/O cost significantly. (**`EXPLAIN SELECT SUM(TotalAmount) FROM Sales WHERE SaleDate >= '2026-04-01' AND SaleDate < '2026-05-01'`**)

5 Data Integrity and Security

5.1 Constraints and Data Validation

Schema-level constraints enforce data accuracy throughout the system. `UNIQUE` constraints were applied to phone numbers to prevent duplicate client profiles, while `CHECK` constraints prevent the system from accepting impossible values (e.g., negative prices or stock quantities).

5.2 Access Control and Security Measures

We implemented two database roles based on the principle of least privilege:

- **pharmacy_admin:** Granted `ALL PRIVILEGES` for schema modification and overall system administration.
- **pharmacy_user:** Restricted to Data Manipulation Language (DML) commands (`SELECT`, `INSERT`, `UPDATE`), preventing unauthorized `DROP` or `DELETE` operations.

For system security, user authentication is managed through a dedicated staff table. Passwords are stored as cryptographic hashes (such as bcrypt or SHA-256) instead of plain text. To mitigate SQL injection risks, any future front end application connected to the database will require to use parameterized queries to ensure user input is never concatenated into executable SQL.

6 Results and Discussion

6.1 Testing and Validation

The system was validated by injecting 20 sample records into each table. The 'Process_Sale' transaction successfully updated stock levels in real-time and showed accurate rollbacks when we artificially forced insufficient stock parameters.

6.2 Challenges and Limitations

Our primary challenge was handling database concurrency during checkout process. Without row-level locking, two simultaneous sales could theoretically deduct stock from same inventory count leading to negative values. We resolved this by wrapping checkout logic in strict START TRANSACTION definitions utilizing FOR UPDATE row locks.

7 Conclusion

This Pharmacy Management System successfully meets all project requirements by delivering a stable, fully normalized backend. By using automated triggers, secure transactions, and optimized indexing, this database is well equipped to handle the constraints of real world pharmacy operations. In the future, this project can be easily expanded by integrating a front-end dashboard for inventory and sales data visualization.

8 References

1. Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.